

CS 6650: What I've Found Valuable So Far

Eroniction Presley —

1 The Scaling Mindset: From “Make It Work” to “Make It Scale”

The most transformative concept from this course is thinking about systems *beyond* functional correctness. Before CS 6650, my instinct when something was slow was to optimize the code. Assignment 6 changed that. My Go product search service was already bounded at 100 products per request — there was no code optimization left. CloudWatch showed CPU scaling linearly with load while memory stayed flat at 7.67%. The lesson: **when you see rising CPU alongside stable memory and no algorithmic headroom, the answer is more compute, not better code.** Learning to read metrics as a decision framework was an “aha” moment. This connects to the slides on scalability: *Scale Up* (bigger hardware) is simple but has a ceiling and is a single point of failure. *Scale Out* (more instances) requires more infrastructure but provides fault tolerance and theoretically unlimited capacity.

2 AWS Architecture: Building My Mental Model

One of the most valuable outcomes has been developing intuition for how cloud infrastructure fits together. The diagrams below capture my mental models: how EC2 instances sit within subnets, are protected by security groups (stateful) and network ACLs (stateless), fronted by a target group that the load balancer routes to (left), and the end-to-end request flow from Internet Gateway through ELB to EC2 instances and database, all within a VPC (right). Assignment 5's product API — built against an OpenAPI spec and deployed with Terraform — was where I first assembled these pieces end-to-end, and Assignment 6's horizontal scaling pushed the architecture to 4,000 RPS with zero failures, making these diagrams feel real rather than theoretical.

3 Concurrency: The Root of All Distributed Problems

Assignment 3's thread experiments gave me visceral proof of why concurrency is hard. Watching Go's **fatal error: concurrent map writes** crash 50 goroutines taught me more about thread safety than any textbook. The progression from plain maps (crash) → Mutex (safe but slow) → RWMutex (better for read-heavy) → sync.Map showed that every synchronization strategy trades off something. Locust load testing extended this to the network level — comparing HttpUser vs. FastHttpUser revealed that at 50 users, the bottleneck was the network round trip, not the client library. **Faster tools don't always mean better performance — the bottleneck matters.** Lamport's paper on logical clocks tied this together: in a distributed system, you can't agree on ordering without explicit mechanisms — the same challenge as goroutines racing on a shared map, just scaled across machines.

4 Self-Healing Infrastructure & Resilience

The resilience test in Assignment 6 was the course highlight. I manually killed an ECS task during a 500-user load test, and the system *self-healed*: ALB detected the failure, ECS launched a replacement in under a minute, and Locust reported zero client-visible errors. The component architecture is elegant — ALB distributes traffic, the Target Group monitors health every 30 seconds, and Auto Scaling watches CloudWatch CPU metrics. Each has one job; together they create a system greater than the sum of its parts.

5 Readings: Failure as the Default Assumption

Two readings shaped my thinking most. Chapter 1 of *Distributed Systems for Fun and Profit* introduced the “replication paradox” — replication both causes and solves problems. Chapter 2's CAP theorem made this concrete: once you accept partitions *will* happen, you're choosing between consistency and availability. The Google Distributed Systems paper reinforced this: **design for failure from the start.** The 8 Fallacies of Distributed Computing sound obvious but are easy to violate implicitly. The Heisenbug vs. Bohrbug distinction — that concurrency makes bugs harder to reproduce because observation changes timing — gave a name to debugging frustrations I'd experienced but never articulated.

6 Peer Learning & Piazza

Beyond lectures and assignments, the Piazza discussions added real value. Seeing classmates' different approaches to the same assignments — different load testing strategies, alternative Terraform configura-

tions, varied observations on where bottlenecks appeared — gave me perspective I wouldn't have gotten working alone. For instance, comparing my Assignment 6 results with peers who saw different throughput numbers on similar configurations helped me appreciate how factors like search term distribution, instance warm-up time, and even Locust client-side CPU limits can affect results. The mock interview format also pushed me to articulate *why* I made certain design choices, not just *what* I built — a skill that directly transfers to system design interviews and real engineering discussions.

7 Consensus, 2PC, and Looking Ahead

The Two-Phase Commit lecture showed why distributed transactions are fundamentally harder than local ones. 2PC's blocking weakness — if the coordinator crashes after deciding but before notifying, participants are stuck — motivated Paxos and Raft's majority-based approach. The FLP impossibility result (consensus can't be guaranteed in bounded time with even one crash) was humbling: it's provably hard, not just unsolved. My mental model for scaling is now: (1) **Measure** — identify the bottleneck with metrics. (2) **Can you optimize?** — is there algorithmic headroom? (3) **Scale direction** — CPU-bound and stateless? Scale out. Stateful? Extract state first. (4) **Design for failure** — health checks, auto-replacement, and redundancy from day one. I'm excited for asynchronous systems, partitioning, and data storage tradeoffs ahead.

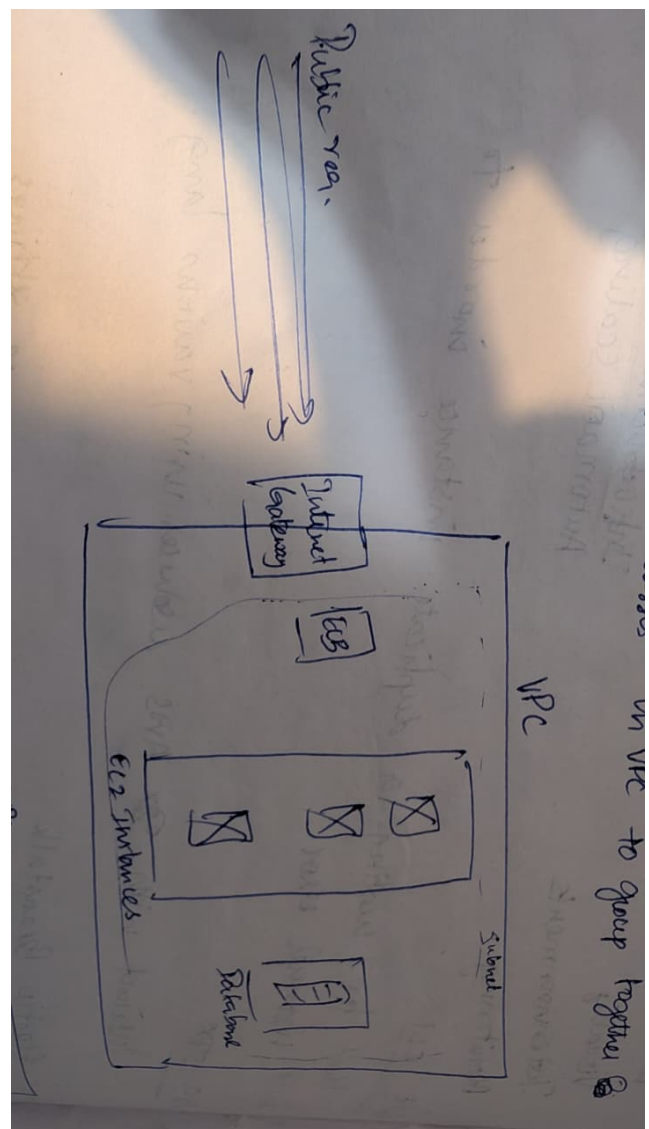
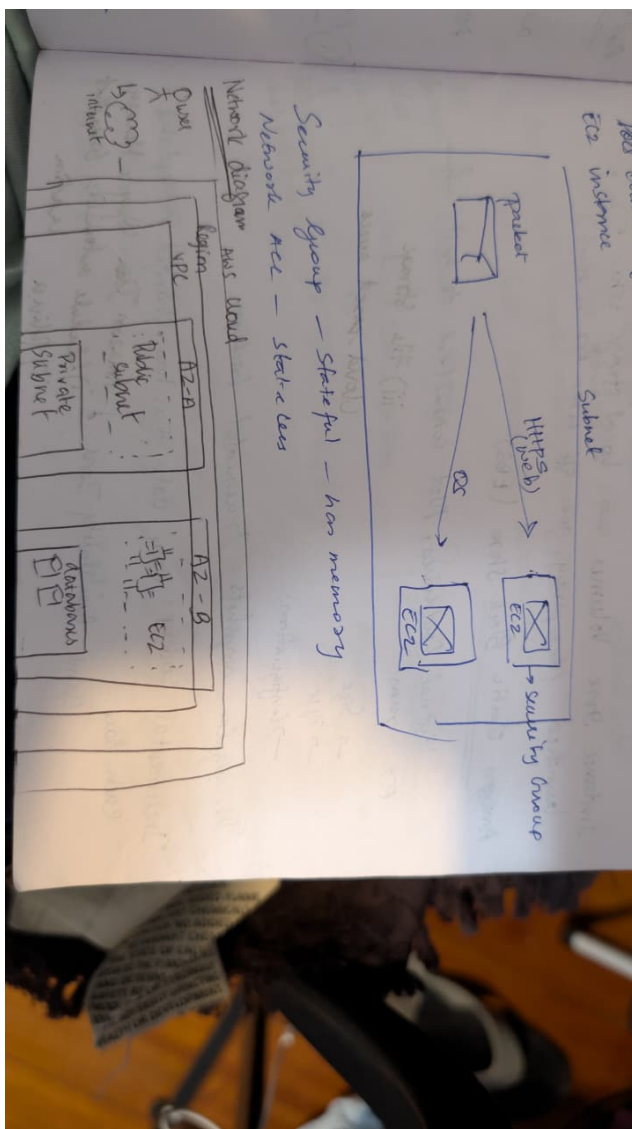


Figure 1: Left: AWS networking — EC2, subnets, security groups, AZs, target groups. Right: Request flow — Internet Gateway → ELB → EC2 → Database within a VPC.